

DATAR- GIONIS- INDYNK- MOTWANI [DGIM] ALGORITHM

Counting Bits

- **Problem:** given a stream of 0's and 1's, be prepared to answer queries of the form “how many 1's in the last k bits?” where $k \leq N$.
- **Obvious solution:** store the most recent N bits.
- But answering the query will take $O(k)$ time.
 - Very possibly too much time.
- And the space requirements can be too great.
 - Especially if there are many streams to be managed in main memory at once, or N is huge.

DGIM Method

- Name refers to the inventors:
 - Datar, Gionis, Indyk, and Motwani.
- Store $O(\log^2 N)$ bits per stream.
- Gives approximate answer, never off by more than 50%.
 - Error factor can be reduced to any fraction > 0 , with more complicated algorithm and proportionally more stored bits.

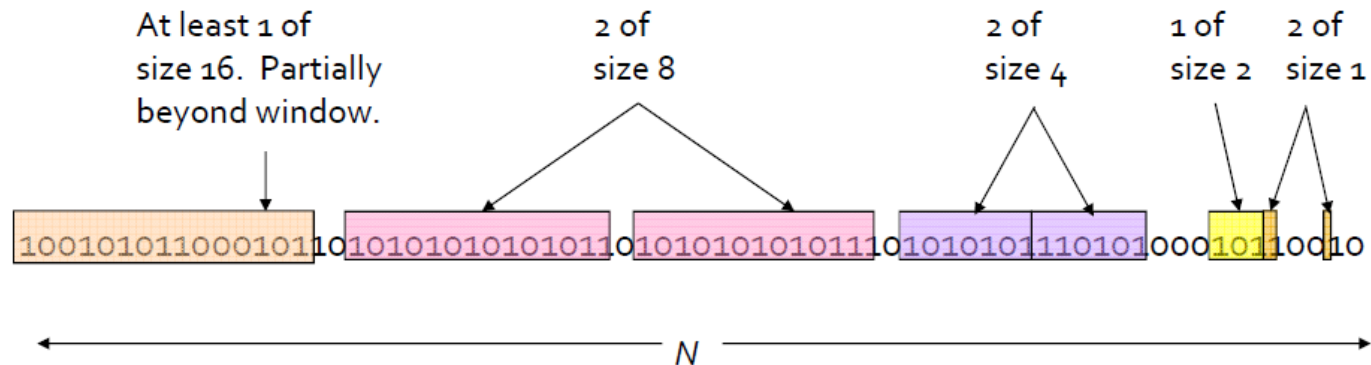
Buckets

- A *bucket* is a segment of the window; it is represented by a record consisting of:
 1. The timestamp of its end [$O(\log N)$ bits].
 2. The number of 1's between its beginning and end [$O(\log \log N)$ bits].
 - Number of 1's = *size* of the bucket.
- **Constraint on buckets:** number of 1's must be a power of 2.
 - That explains the $\log \log N$ in (2).

Representing a Stream by Buckets

- Either one or two buckets with the same power-of-2 number of 1's.
- Buckets do not overlap.
- Buckets are sorted by size.
 - Earlier buckets are not smaller than later buckets.
- Buckets disappear when their end-time is $> N$ time units in the past.

Example: Bucketized Stream



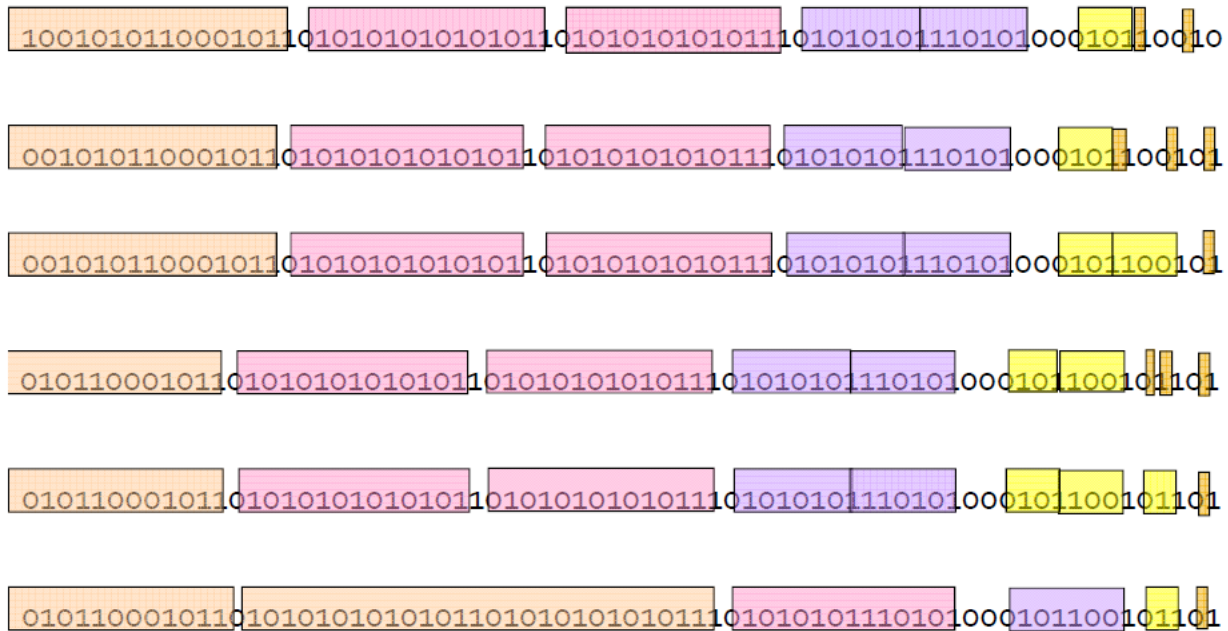
Updating Buckets

- When a new bit comes in, drop the last (oldest) bucket if its end-time is prior to N time units before the current time.
- If the current bit is 0, no other changes are needed.

Updating Buckets: Input = 1

- If the current bit is 1:
 1. Create a new bucket of size 1, for just this bit.
 - End timestamp = current time.
 2. If there are now three buckets of size 1, combine the oldest two into a bucket of size 2.
 3. If there are now three buckets of size 2, combine the oldest two into a bucket of size 4.
 4. And so on ...

Example



Querying

- To estimate the number of 1's in the most recent $k \leq N$ bits:
 1. Restrict your attention to only those buckets whose end time stamp is at most k bits in the past.
 2. Sum the sizes of all these buckets but the oldest.
 3. Add half the size of the oldest bucket.
- **Remember:** we don't know how many 1's of the last bucket are still within the window.

Error Bound

- Suppose the oldest bucket within range has size 2^i .
- Then by assuming 2^{i-1} of its 1's are still within the window, we make an error of at most 2^{i-1} .
- Since there is at least one bucket of each of the sizes less than 2^i , and at least 1 from the oldest bucket, the true sum is no less than 2^i .
- Thus, error at most 50%.

Numericals

DGIM ALGORITHM

- **A**lgorithm that uses $O(\log^2 N)$ bits to represent a window of N bit, and allows to estimate the number of 1's in the window with an error of no more than 50%.

DGIM ALGORITHM

- **E**ach bit stream has a timestamp, for the position at which it arrives.
- **T**he first bit has timestamp 1, the second bit has timestamp 2, and so on.
- **T**he positions are recognized with the window size N .
- **T**he timestamp is represented with modulo N , and are represented as $\log_2 N$ bits.

DGIM ALGORITHM

- **T**he windows are divided into buckets consisting of
 - **T**he timestamp at its right end.
 - **T**he number of ones must be in the power of 2, which are referred to as size of bucket.
 - E.g. 1001011 -> the bucket size is 4.

RULES FOR FORMING BUCKET

- **T**he right side of the bucket should always start with 1.
- **E**very bucket should have at least one 1, else no bucket can be formed.
- **A**ll bucket sizes should be a power of 2.
- **T**he buckets cannot decrease in size as we move to the left.

EXAMPLE

- **T**he input stream bit 101011000101110110010110
- **N** = 24
- **E**stimate the number of buckets and 1's

EXAMPLE

➤ The input stream bit 101011000101110110010110

101011 000 10111 0 11 00101 1 0

UPDATING BUCKETS

- **W**hen a new bit comes in, drop the last bucket if its end-time is prior to N time units before the current time.
- **I**f the current bit is 0, no other changes are needed.

UPDATING BUCKETS

➤ If the current bit is 1

- Create a new bucket of size 1, for just this bit.

End timestamp = current time.

- If there are now three buckets of size 1, combine the oldest two into a bucket of size 2.

1010110001011101100101101

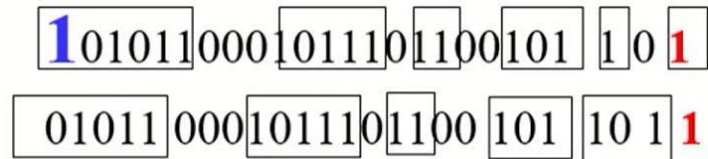
UPDATING BUCKETS

➤ If the current bit is 1

- Create a new bucket of size 1, for just this bit.

End timestamp = current time.

- If there are now three buckets of size 1, combine the oldest two into a bucket of size 2.



EXAMPLE

- The input stream bit 101011000101110110010110
- The new input stream of 10101011 to be added.

EXAMPLE

- **T**he input stream bit 101011000101110110010110
- **T**he new input stream of 10101011 to be added.
- **T**he bit stream is always added to the right side of previous data.

EXAMPLE

- The input stream bit 101011000101110110010110
- The stream now becomes

101011000101110110010110 10101011

↑
Added stream bit

EXAMPLE

- The new input stream bit
10101100010111011001011010101011
- $N = 32$
- The buckets are formed as

10101100010111

011001011010101011


Bucket of size 8

EXAMPLE

- The new input stream bit
10101100010111011001011010101011
- $N = 32$
- The buckets are formed as
10101100010111 0 1100101 1010101011
 ↑
 Bucket of size 4

EXAMPLE

- The buckets are formed as

↑
Bucket of size 2

EXAMPLE

- **The new input stream bit**
10101100010111011001011010101011
- **N = 32**
- **The buckets are formed as**

10101100010111 01100101 101 0101 011

Bucket of size 2

EXAMPLE

- The new input stream bit

10101100010111011001011010101011

- $N = 32$

- The buckets are formed as

10101100010111 01100101 101 0101 01 1

↑
Bucket of size 1

EXAMPLE

➤ The new input stream bit
10101100010111011001011010101011

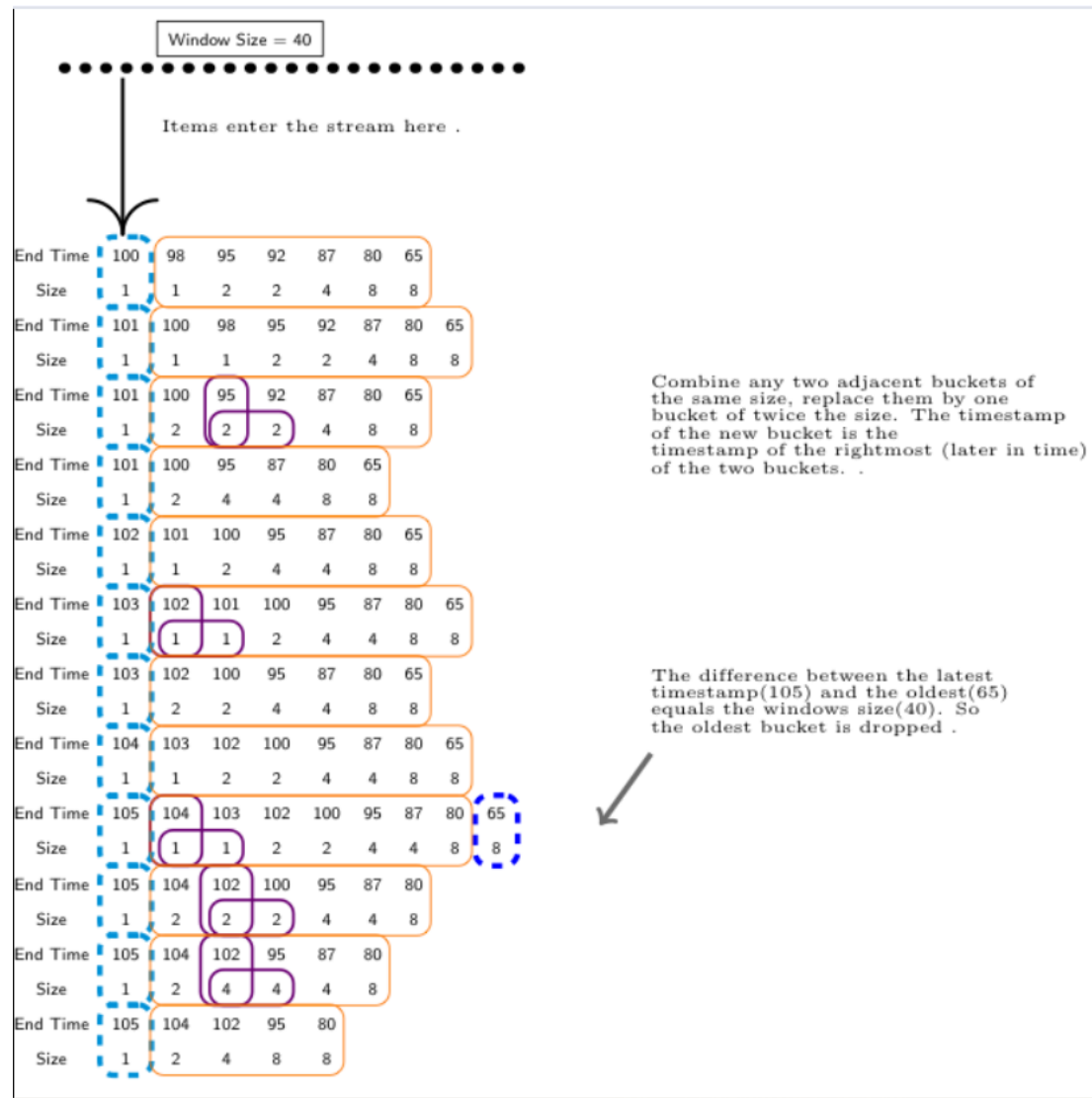
➤ $N = 32$

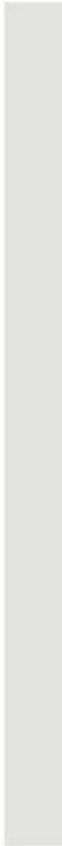
➤ The buckets are formed as

10101100010111 01100101 101 0101 011

↑
Bucket of size 1

Example 2:





Thank you